

Tello

Python wrapper to interact with the Ryze Tello drone using the official Tello api. Tello API documentation:

[1.3, 2.0 with EDU-only commands](#)

Source code in `djitellopy/tello.py`

```
27 @enforce_types
28 class Tello:
29     """Python wrapper to interact with the Ryze Tello drone using the official Tello
30     api.
31     Tello API documentation:
32     [1.3](https://dl-
33     cdn.ryzerobotics.com/downloads/tello/20180910/Tello%20SDK%20Documentation%20EN_1.3.pdf),
34     [2.0 with EDU-only commands](https://dl-
35     cdn.ryzerobotics.com/downloads/Tello/Tello%20SDK%202.0%20User%20Guide.pdf)
36     """
37     # Send and receive commands, client socket
38     RESPONSE_TIMEOUT = 7 # in seconds
39     TAKEOFF_TIMEOUT = 20 # in seconds
40     FRAME_GRAB_TIMEOUT = 5
41     TIME_BTW_COMMANDS = 0.1 # in seconds
42     TIME_BTW_RC_CONTROL_COMMANDS = 0.001 # in seconds
43     RETRY_COUNT = 3 # number of retries after a failed command
44     TELLO_IP = '192.168.10.1' # Tello IP address
45
46     # Video stream, server socket
47     VS_UDP_IP = '0.0.0.0'
48     VS_UDP_PORT = 11111
49
50     CONTROL_UDP_PORT = 8889
51     STATE_UDP_PORT = 8890
52
53     # Constants for video settings
54     BITRATE_AUTO = 0
55     BITRATE_1MBPS = 1
56     BITRATE_2MBPS = 2
57     BITRATE_3MBPS = 3
58     BITRATE_4MBPS = 4
59     BITRATE_5MBPS = 5
60     RESOLUTION_480P = 'low'
61     RESOLUTION_720P = 'high'
62     FPS_5 = 'low'
63     FPS_15 = 'middle'
64     FPS_30 = 'high'
65     CAMERA_FORWARD = 0
66     CAMERA_DOWNWARD = 1
67
68     # Set up logger
69     HANDLER = logging.StreamHandler()
70     FORMATTER = logging.Formatter('[%(levelname)s] %(filename)s - %(lineno)d - %
```

```

65 (message)s')
66     HANDLER.setFormatter(FORMATTER)
67
68     LOGGER = logging.getLogger('djitelloy')
69     LOGGER.addHandler(HANDLER)
70     LOGGER.setLevel(logging.INFO)
71     # Use Tello.LOGGER.setLevel(logging.<LEVEL>) in YOUR CODE
72     # to only receive logs of the desired level and higher
73
74     # Conversion functions for state protocol fields
75     INT_STATE_FIELDS = (
76         # Tello EDU with mission pads enabled only
77         'mid', 'x', 'y', 'z',
78         # 'mpy': (custom format 'x,y,z')
79         # Common entries
80         'pitch', 'roll', 'yaw',
81         'vgx', 'vgy', 'vgz',
82         'templ', 'temph',
83         'tof', 'h', 'bat', 'time'
84     )
85     FLOAT_STATE_FIELDS = ('baro', 'agx', 'agy', 'agz')
86
87     state_field_converters: Dict[str, Union[Type[int], Type[float]]]
88     state_field_converters = {key : int for key in INT_STATE_FIELDS}
89     state_field_converters.update({key : float for key in FLOAT_STATE_FIELDS})
90
91     # VideoCapture object
92     background_frame_read: Optional['BackgroundFrameRead'] = None
93
94     stream_on = False
95     is_flying = False
96
97     def __init__(self,
98                 host=TELLO_IP,
99                 retry_count=RETRY_COUNT,
100                 vs_udp=VS_UDP_PORT):
101
102         global threads_initialized, client_socket, drones
103
104         self.address = (host, Tello.CONTROL_UDP_PORT)
105         self.stream_on = False
106         self.retry_count = retry_count
107         self.last_received_command_timestamp = time.time()
108         self.last_rc_control_timestamp = time.time()
109
110         if not threads_initialized:
111             # Run Tello command responses UDP receiver on background
112             client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
113             client_socket.bind(("", Tello.CONTROL_UDP_PORT))
114             response_receiver_thread = Thread(target=Tello.udp_response_receiver)
115             response_receiver_thread.daemon = True
116             response_receiver_thread.start()
117
118             # Run state UDP receiver on background
119             state_receiver_thread = Thread(target=Tello.udp_state_receiver)
120             state_receiver_thread.daemon = True
121             state_receiver_thread.start()
122
123             threads_initialized = True

```

```

116         drones[host] = {'responses': [], 'state': {}}
117
118         self.LOGGER.info("Tello instance was initialized. Host: '{}'. Port:
119 '{}'.format(host, Tello.CONTROL_UDP_PORT))
120
121         self.vs_udp_port = vs_udp
122
123     def change_vs_udp(self, udp_port):
124         """Change the UDP Port for sending video feed from the drone.
125         """
126         self.vs_udp_port = udp_port
127         self.send_control_command(f'port 8890 {self.vs_udp_port}')
128
129     def get_own_udp_object(self):
130         """Get own object from the global drones dict. This object is filled
131         with responses and state information by the receiver threads.
132         Internal method, you normally wouldn't call this yourself.
133         """
134         global drones
135
136         host = self.address[0]
137         return drones[host]
138
139     @staticmethod
140     def udp_response_receiver():
141         """Setup drone UDP receiver. This method listens for responses of Tello.
142         Must be run from a background thread in order to not block the main thread.
143         Internal method, you normally wouldn't call this yourself.
144         """
145         while True:
146             try:
147                 data, address = client_socket.recvfrom(1024)
148
149                 address = address[0]
150                 Tello.LOGGER.debug('Data received from {} at
151 client_socket'.format(address))
152
153                 if address not in drones:
154                     continue
155
156                 drones[address]['responses'].append(data)
157
158             except Exception as e:
159                 Tello.LOGGER.error(e)
160                 break
161
162     @staticmethod
163     def udp_state_receiver():
164         """Setup state UDP receiver. This method listens for state information from
165         Tello. Must be run from a background thread in order to not block
166         the main thread.
167         Internal method, you normally wouldn't call this yourself.
168         """
169         state_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
170         state_socket.bind(("", Tello.STATE_UDP_PORT))
171
172         while True:
173             try:
174                 data, address = state_socket.recvfrom(1024)

```

```

167         address = address[0]
168         Tello.LOGGER.debug('Data received from {} at
169 state_socket'.format(address))
170
171         if address not in drones:
172             continue
173
174         data = data.decode('ASCII')
175         drones[address]['state'] = Tello.parse_state(data)
176
177     except Exception as e:
178         Tello.LOGGER.error(e)
179         break
180
181 @staticmethod
182 def parse_state(state: str) -> Dict[str, Union[int, float, str]]:
183     """Parse a state line to a dictionary
184     Internal method, you normally wouldn't call this yourself.
185     """
186     state = state.strip()
187     Tello.LOGGER.debug('Raw state data: {}'.format(state))
188
189     if state == 'ok':
190         return {}
191
192     state_dict = {}
193     for field in state.split(';'):
194         split = field.split(':')
195         if len(split) < 2:
196             continue
197
198         key = split[0]
199         value: Union[int, float, str] = split[1]
200
201         if key in Tello.state_field_converters:
202             num_type = Tello.state_field_converters[key]
203             try:
204                 value = num_type(value)
205             except ValueError as e:
206                 Tello.LOGGER.debug('Error parsing state value for {}: {} to {}'.format(key, value, num_type))
207                 Tello.LOGGER.error(e)
208                 continue
209
210         state_dict[key] = value
211
212     return state_dict
213
214 def get_current_state(self) -> dict:
215     """Call this function to attain the state of the Tello. Returns a dict
216     with all fields.
217     Internal method, you normally wouldn't call this yourself.
218     """
219     return self.get_own_udp_object()['state']
220
221 def get_state_field(self, key: str):
222     """Get a specific sate field by name.
223     Internal method, you normally wouldn't call this yourself.
224     """

```

```
218         state = self.get_current_state()
219
220         if key in state:
221             return state[key]
222         else:
223             raise TelloException('Could not get state property: {}'.format(key))
224
225     def get_mission_pad_id(self) -> int:
226         """Mission pad ID of the currently detected mission pad
227         Only available on Tello EDUs after calling enable_mission_pads
228         Returns:
229             int: -1 if none is detected, else 1-8
230         """
231         return self.get_state_field('mid')
232
233     def get_mission_pad_distance_x(self) -> int:
234         """X distance to current mission pad
235         Only available on Tello EDUs after calling enable_mission_pads
236         Returns:
237             int: distance in cm
238         """
239         return self.get_state_field('x')
240
241     def get_mission_pad_distance_y(self) -> int:
242         """Y distance to current mission pad
243         Only available on Tello EDUs after calling enable_mission_pads
244         Returns:
245             int: distance in cm
246         """
247         return self.get_state_field('y')
248
249     def get_mission_pad_distance_z(self) -> int:
250         """Z distance to current mission pad
251         Only available on Tello EDUs after calling enable_mission_pads
252         Returns:
253             int: distance in cm
254         """
255         return self.get_state_field('z')
256
257     def get_pitch(self) -> int:
258         """Get pitch in degree
259         Returns:
260             int: pitch in degree
261         """
262         return self.get_state_field('pitch')
263
264     def get_roll(self) -> int:
265         """Get roll in degree
266         Returns:
267             int: roll in degree
268         """
269         return self.get_state_field('roll')
270
271     def get_yaw(self) -> int:
272         """Get yaw in degree
273         Returns:
274             int: yaw in degree
275         """
276         return self.get_state_field('yaw')
```

```
269     def get_speed_x(self) -> int:
270         """X-Axis Speed
271         Returns:
272             int: speed
273         """
274         return self.get_state_field('vgx')
275
276     def get_speed_y(self) -> int:
277         """Y-Axis Speed
278         Returns:
279             int: speed
280         """
281         return self.get_state_field('vgy')
282
283     def get_speed_z(self) -> int:
284         """Z-Axis Speed
285         Returns:
286             int: speed
287         """
288         return self.get_state_field('vgz')
289
290     def get_acceleration_x(self) -> float:
291         """X-Axis Acceleration
292         Returns:
293             float: acceleration
294         """
295         return self.get_state_field('agx')
296
297     def get_acceleration_y(self) -> float:
298         """Y-Axis Acceleration
299         Returns:
300             float: acceleration
301         """
302         return self.get_state_field('agy')
303
304     def get_acceleration_z(self) -> float:
305         """Z-Axis Acceleration
306         Returns:
307             float: acceleration
308         """
309         return self.get_state_field('agz')
310
311     def get_lowest_temperature(self) -> int:
312         """Get lowest temperature
313         Returns:
314             int: lowest temperature (°C)
315         """
316         return self.get_state_field('templ')
317
318     def get_highest_temperature(self) -> int:
319         """Get highest temperature
320         Returns:
321             float: highest temperature (°C)
322         """
323         return self.get_state_field('temph')
324
325     def get_temperature(self) -> float:
326         """Get average temperature
327         Returns:
328             float: average temperature (°C)
```

```

320         """
321         templ = self.get_lowest_temperature()
322         temph = self.get_highest_temperature()
323         return (templ + temph) / 2
324
325     def get_height(self) -> int:
326         """Get current height in cm
327         Returns:
328             int: height in cm
329         """
330         return self.get_state_field('h')
331
332     def get_distance_to_f(self) -> int:
333         """Get current distance value from TOF in cm
334         Returns:
335             int: TOF distance in cm
336         """
337         return self.get_state_field('tof')
338
339     def get_barometer(self) -> int:
340         """Get current barometer measurement in cm
341         This resembles the absolute height.
342         See https://en.wikipedia.org/wiki/Altimeter
343         Returns:
344             int: barometer measurement in cm
345         """
346         return self.get_state_field('baro') * 100
347
348     def get_flight_time(self) -> int:
349         """Get the time the motors have been active in seconds
350         Returns:
351             int: flight time in s
352         """
353         return self.get_state_field('time')
354
355     def get_battery(self) -> int:
356         """Get current battery percentage
357         Returns:
358             int: 0-100
359         """
360         return self.get_state_field('bat')
361
362     def get_udp_video_address(self) -> str:
363         """Internal method, you normally wouldn't call this yourself.
364         """
365         address_schema = 'udp://@{ip}:{port}' # + '?overrun_nonfatal=1&fifo_size=5000'
366         address = address_schema.format(ip=self.VS_UDP_IP, port=self.vs_udp_port)
367         return address
368
369     def get_frame_read(self, with_queue = False, max_queue_len = 32) ->
370     'BackgroundFrameRead':
371         """Get the BackgroundFrameRead object from the camera drone. Then, you just need
372         to call
373         backgroundFrameRead.frame to get the actual frame received by the drone.
374         Returns:
375             BackgroundFrameRead
376         """
377         if self.background_frame_read is None:
378             address = self.get_udp_video_address()
379             self.background_frame_read = BackgroundFrameRead(self, address, with_queue,

```

```

371     max_queue_len)
372         self.background_frame_read.start()
373         return self.background_frame_read
374
375     def send_command_with_return(self, command: str, timeout: int = RESPONSE_TIMEOUT) ->
376     str:
377         """Send command to Tello and wait for its response.
378         Internal method, you normally wouldn't call this yourself.
379         Return:
380             bool/str: str with response text on success, False when unsuccessful.
381             """
382         # Commands very consecutive makes the drone not respond to them.
383         # So wait at least self.TIME_BTW_COMMANDS seconds
384         diff = time.time() - self.last_received_command_timestamp
385         if diff < self.TIME_BTW_COMMANDS:
386             self.LOGGER.debug('Waiting {} seconds to execute command:
387             {}...'.format(diff, command))
388             time.sleep(diff)
389
390         self.LOGGER.info("Send command: '{}'.format(command))
391         timestamp = time.time()
392
393         client_socket.sendto(command.encode('utf-8'), self.address)
394
395         responses = self.get_own_udp_object()['responses']
396
397         while not responses:
398             if time.time() - timestamp > timeout:
399                 message = "Aborting command '{}'. Did not receive a response after {}
400                 seconds".format(command, timeout)
401                 self.LOGGER.warning(message)
402                 return message
403                 time.sleep(0.1) # Sleep during send command
404
405         self.last_received_command_timestamp = time.time()
406
407         first_response = responses.pop(0) # first datum from socket
408         try:
409             response = first_response.decode("utf-8")
410         except UnicodeDecodeError as e:
411             self.LOGGER.error(e)
412             return "response decode error"
413         response = response.rstrip("\r\n")
414
415         self.LOGGER.info("Response {}: '{}'.format(command, response))
416         return response
417
418     def send_command_without_return(self, command: str):
419         """Send command to Tello without expecting a response.
420         Internal method, you normally wouldn't call this yourself.
421         """
422         # Commands very consecutive makes the drone not respond to them. So wait at
423         least self.TIME_BTW_COMMANDS seconds
424
425         self.LOGGER.info("Send command (no response expected): '{}'.format(command))
426         client_socket.sendto(command.encode('utf-8'), self.address)
427
428     def send_control_command(self, command: str, timeout: int = RESPONSE_TIMEOUT) ->
429     bool:
430         """Send control command to Tello and wait for its response.

```

```

422         Internal method, you normally wouldn't call this yourself.
423         """
424         response = "max retries exceeded"
425         for i in range(0, self.retry_count):
426             response = self.send_command_with_return(command, timeout=timeout)
427
428             if 'ok' in response.lower():
429                 return True
430
431             self.LOGGER.debug("Command attempt #{i} failed for command: '{command}'".format(i,
432 command))
433
434             self.raise_result_error(command, response)
435             return False # never reached
436
437     def send_read_command(self, command: str) -> str:
438         """Send given command to Tello and wait for its response.
439         Internal method, you normally wouldn't call this yourself.
440         """
441
442         response = self.send_command_with_return(command)
443
444         try:
445             response = str(response)
446         except TypeError as e:
447             self.LOGGER.error(e)
448
449         if any(word in response for word in ('error', 'ERROR', 'False')):
450             self.raise_result_error(command, response)
451             return "Error: this code should never be reached"
452
453         return response
454
455     def send_read_command_int(self, command: str) -> int:
456         """Send given command to Tello and wait for its response.
457         Parses the response to an integer
458         Internal method, you normally wouldn't call this yourself.
459         """
460
461         response = self.send_read_command(command)
462         return int(response)
463
464     def send_read_command_float(self, command: str) -> float:
465         """Send given command to Tello and wait for its response.
466         Parses the response to an integer
467         Internal method, you normally wouldn't call this yourself.
468         """
469
470         response = self.send_read_command(command)
471         return float(response)
472
473     def raise_result_error(self, command: str, response: str) -> bool:
474         """Used to raise an error after an unsuccessful command
475         Internal method, you normally wouldn't call this yourself.
476         """
477
478         tries = 1 + self.retry_count
479         raise TelloException("Command '{command}' was unsuccessful for {tries} tries. Latest
480 response:\t'{response}'"
481                             .format(command, tries, response))
482
483     def connect(self, wait_for_state=True):
484         """Enter SDK mode. Call this before any of the control functions.

```

```

473         """
474         self.send_control_command("command")
475
476         if wait_for_state:
477             REPS = 20
478             for i in range(REPS):
479                 if self.get_current_state():
480                     t = i / REPS # in seconds
481                     Tello.LOGGER.debug("''.connect()' received first state packet after
482 {} seconds".format(t))
483                     break
484                     time.sleep(1 / REPS)
485
486             if not self.get_current_state():
487                 raise TelloException('Did not receive a state packet from the Tello')
488
489     def send_keepalive(self):
490         """Send a keepalive packet to prevent the drone from landing after 15s
491         """
492         self.send_control_command("keepalive")
493
494     def turn_motor_on(self):
495         """Turn on motors without flying (mainly for cooling)
496         """
497         self.send_control_command("motoron")
498
499     def turn_motor_off(self):
500         """Turns off the motor cooling mode
501         """
502         self.send_control_command("motoroff")
503
504     def initiate_throw_takeoff(self):
505         """Allows you to take off by throwing your drone within 5 seconds of this
506         command
507         """
508         self.send_control_command("throwfly")
509         self.is_flying = True
510
511     def takeoff(self):
512         """Automatic takeoff.
513         """
514         # Something it takes a looooooot of time to take off and return a succesful
515         takeoff.
516         # So we better wait. Otherwise, it would give us an error on the following
517         calls.
518         self.send_control_command("takeoff", timeout=Tello.TAKEOFF_TIMEOUT)
519         self.is_flying = True
520
521     def land(self):
522         """Automatic landing.
523         """
524         self.send_control_command("land")
525         self.is_flying = False
526
527     def streamon(self):
528         """Turn on video streaming. Use `tello.get_frame_read` afterwards.
529         Video Streaming is supported on all tellos when in AP mode (i.e.
530         when your computer is connected to Tello-XXXXXX WiFi network).
531         Currently Tello EDUs do not support video streaming while connected
532         to a WiFi-network.

```

```
524
525     !!! Note:
526         If the response is 'Unknown command' you have to update the Tello
527         firmware. This can be done using the official Tello app.
528     """
529     self.send_control_command("streamon")
530     self.stream_on = True
531
532     def streamoff(self):
533         """Turn off video streaming.
534         """
535         self.send_control_command("streamoff")
536         self.stream_on = False
537
538         if self.background_frame_read is not None:
539             self.background_frame_read.stop()
540             self.background_frame_read = None
541
542     def emergency(self):
543         """Stop all motors immediately.
544         """
545         self.send_command_without_return("emergency")
546         self.is_flying = False
547
548     def move(self, direction: str, x: int):
549         """Tello fly up, down, left, right, forward or back with distance x cm.
550         Users would normally call one of the move_x functions instead.
551         Arguments:
552             direction: up, down, left, right, forward or back
553             x: 20-500
554         """
555         self.send_control_command("{} {} {}".format(direction, x))
556
557     def move_up(self, x: int):
558         """Fly x cm up.
559         Arguments:
560             x: 20-500
561         """
562         self.move("up", x)
563
564     def move_down(self, x: int):
565         """Fly x cm down.
566         Arguments:
567             x: 20-500
568         """
569         self.move("down", x)
570
571     def move_left(self, x: int):
572         """Fly x cm left.
573         Arguments:
574             x: 20-500
575         """
576         self.move("left", x)
577
578     def move_right(self, x: int):
579         """Fly x cm right.
580         Arguments:
581             x: 20-500
582         """
583         self.move("right", x)
```

```
575
576     def move_forward(self, x: int):
577         """Fly x cm forward.
578         Arguments:
579             x: 20-500
580             """
581         self.move("forward", x)
582
583     def move_back(self, x: int):
584         """Fly x cm backwards.
585         Arguments:
586             x: 20-500
587             """
588         self.move("back", x)
589
590     def rotate_clockwise(self, x: int):
591         """Rotate x degree clockwise.
592         Arguments:
593             x: 1-360
594             """
595         self.send_control_command("cw {}".format(x))
596
597     def rotate_counter_clockwise(self, x: int):
598         """Rotate x degree counter-clockwise.
599         Arguments:
600             x: 1-360
601             """
602         self.send_control_command("ccw {}".format(x))
603
604     def flip(self, direction: str):
605         """Do a flip maneuver.
606         Users would normally call one of the flip_x functions instead.
607         Arguments:
608             direction: l (left), r (right), f (forward) or b (back)
609             """
610         self.send_control_command("flip {}".format(direction))
611
612     def flip_left(self):
613         """Flip to the left.
614         """
615         self.flip("l")
616
617     def flip_right(self):
618         """Flip to the right.
619         """
620         self.flip("r")
621
622     def flip_forward(self):
623         """Flip forward.
624         """
625         self.flip("f")
626
627     def flip_back(self):
628         """Flip backwards.
629         """
630         self.flip("b")
631
632     def go_xyz_speed(self, x: int, y: int, z: int, speed: int):
633         """Fly to x y z relative to the current position.
634         Speed defines the traveling speed in cm/s.
```

```

626         Arguments:
627             x: -500-500
628             y: -500-500
629             z: -500-500
630             speed: 10-100
631         """
632         cmd = 'go {} {} {} {}'.format(x, y, z, speed)
633         self.send_control_command(cmd)
634
635     def curve_xyz_speed(self, x1: int, y1: int, z1: int, x2: int, y2: int, z2: int,
636 speed: int):
637         """Fly to x2 y2 z2 in a curve via x2 y2 z2. Speed defines the traveling speed in
638 cm/s.
639
640         - Both points are relative to the current position
641         - The current position and both points must form a circle arc.
642         - If the arc radius is not within the range of 0.5-10 meters, it raises an
643 Exception
644         - x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can
645 both be 0.
646
647         Arguments:
648             x1: -500-500
649             x2: -500-500
650             y1: -500-500
651             y2: -500-500
652             z1: -500-500
653             z2: -500-500
654             speed: 10-60
655         """
656         cmd = 'curve {} {} {} {} {} {} {}'.format(x1, y1, z1, x2, y2, z2, speed)
657         self.send_control_command(cmd)
658
659     def go_xyz_speed_mid(self, x: int, y: int, z: int, speed: int, mid: int):
660         """Fly to x y z relative to the mission pad with id mid.
661         Speed defines the traveling speed in cm/s.
662         Arguments:
663             x: -500-500
664             y: -500-500
665             z: -500-500
666             speed: 10-100
667             mid: 1-8
668         """
669         cmd = 'go {} {} {} {} m{}'.format(x, y, z, speed, mid)
670         self.send_control_command(cmd)
671
672     def curve_xyz_speed_mid(self, x1: int, y1: int, z1: int, x2: int, y2: int, z2: int,
673 speed: int, mid: int):
674         """Fly to x2 y2 z2 in a curve via x2 y2 z2. Speed defines the traveling speed in
675 cm/s.
676
677         - Both points are relative to the mission pad with id mid.
678         - The current position and both points must form a circle arc.
679         - If the arc radius is not within the range of 0.5-10 meters, it raises an
680 Exception
681         - x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can
682 both be 0.
683
684         Arguments:
685             x1: -500-500

```

```

677         y1: -500-500
678         z1: -500-500
679         x2: -500-500
680         y2: -500-500
681         z2: -500-500
682         speed: 10-60
683         mid: 1-8
684         """
685         cmd = 'curve {} {} {} {} {} {} {} m{}'.format(x1, y1, z1, x2, y2, z2, speed,
686         mid)
687         self.send_control_command(cmd)
688
689     def go_xyz_speed_yaw_mid(self, x: int, y: int, z: int, speed: int, yaw: int, mid1:
690     int, mid2: int):
691         """Fly to x y z relative to mid1.
692         Then fly to 0 0 z over mid2 and rotate to yaw relative to mid2's rotation.
693         Speed defines the traveling speed in cm/s.
694         Arguments:
695         x: -500-500
696         y: -500-500
697         z: -500-500
698         speed: 10-100
699         yaw: -360-360
700         mid1: 1-8
701         mid2: 1-8
702         """
703         cmd = 'jump {} {} {} {} {} {} m{} m{}'.format(x, y, z, speed, yaw, mid1, mid2)
704         self.send_control_command(cmd)
705
706     def enable_mission_pads(self):
707         """Enable mission pad detection
708         """
709         self.send_control_command("mon")
710
711     def disable_mission_pads(self):
712         """Disable mission pad detection
713         """
714         self.send_control_command("moff")
715
716     def set_mission_pad_detection_direction(self, x):
717         """Set mission pad detection direction. enable_mission_pads needs to be
718         called first. When detecting both directions detecting frequency is 10Hz,
719         otherwise the detection frequency is 20Hz.
720         Arguments:
721         x: 0 downwards only, 1 forwards only, 2 both directions
722         """
723         self.send_control_command("mdirection {}".format(x))
724
725     def set_speed(self, x: int):
726         """Set speed to x cm/s.
727         Arguments:
728         x: 10-100
729         """
730         self.send_control_command("speed {}".format(x))
731
732     def send_rc_control(self, left_right_velocity: int, forward_backward_velocity: int,
733     up_down_velocity: int,
734     yaw_velocity: int):
735         """Send RC control via four channels. Command is sent every
736         self.TIME_BTW_RC_CONTROL_COMMANDS seconds.

```

```

728         Arguments:
729             left_right_velocity: -100~100 (left/right)
730             forward_backward_velocity: -100~100 (forward/backward)
731             up_down_velocity: -100~100 (up/down)
732             yaw_velocity: -100~100 (yaw)
733         """
734         def clamp100(x: int) -> int:
735             return max(-100, min(100, x))
736
737         if time.time() - self.last_rc_control_timestamp >
738 self.TIME_BTW_RC_CONTROL_COMMANDS:
739             self.last_rc_control_timestamp = time.time()
740             cmd = 'rc {} {} {} {}'.format(
741                 clamp100(left_right_velocity),
742                 clamp100(forward_backward_velocity),
743                 clamp100(up_down_velocity),
744                 clamp100(yaw_velocity)
745             )
746             self.send_command_without_return(cmd)
747
748         def set_wifi_credentials(self, ssid: str, password: str):
749             """Set the Wi-Fi SSID and password. The Tello will reboot afterwards.
750             """
751             cmd = 'wifi {} {}'.format(ssid, password)
752             self.send_control_command(cmd)
753
754         def connect_to_wifi(self, ssid: str, password: str):
755             """Connects to the Wi-Fi with SSID and password.
756             After this command the tello will reboot.
757             Only works with Tello EDUs.
758             """
759             cmd = 'ap {} {}'.format(ssid, password)
760             self.send_control_command(cmd)
761
762         def set_network_ports(self, state_packet_port: int, video_stream_port: int):
763             """Sets the ports for state packets and video streaming
764             While you can use this command to reconfigure the Tello this library currently
765             does not support
766             non-default ports (TODO!)
767             """
768             cmd = 'port {} {}'.format(state_packet_port, video_stream_port)
769             self.send_control_command(cmd)
770
771         def reboot(self):
772             """Reboots the drone
773             """
774             self.send_command_without_return('reboot')
775
776         def set_video_bitrate(self, bitrate: int):
777             """Sets the bitrate of the video stream
778             Use one of the following for the bitrate argument:
779             Tello.BITRATE_AUTO
780             Tello.BITRATE_1MBPS
781             Tello.BITRATE_2MBPS
782             Tello.BITRATE_3MBPS
783             Tello.BITRATE_4MBPS
784             Tello.BITRATE_5MBPS
785             """
786             cmd = 'setbitrate {}'.format(bitrate)
787             self.send_control_command(cmd)

```

```

779
780     def set_video_resolution(self, resolution: str):
781         """Sets the resolution of the video stream
782         Use one of the following for the resolution argument:
783             Tello.RESOLUTION_480P
784             Tello.RESOLUTION_720P
785         """
786         cmd = 'setresolution {}'.format(resolution)
787         self.send_control_command(cmd)
788
789     def set_video_fps(self, fps: str):
790         """Sets the frames per second of the video stream
791         Use one of the following for the fps argument:
792             Tello.FPS_5
793             Tello.FPS_15
794             Tello.FPS_30
795         """
796         cmd = 'setfps {}'.format(fps)
797         self.send_control_command(cmd)
798
799     def set_video_direction(self, direction: int):
800         """Selects one of the two cameras for video streaming
801         The forward camera is the regular 1080x720 color camera
802         The downward camera is a grey-only 320x240 IR-sensitive camera
803         Use one of the following for the direction argument:
804             Tello.CAMERA_FORWARD
805             Tello.CAMERA_DOWNWARD
806         """
807         cmd = 'downvision {}'.format(direction)
808         self.send_control_command(cmd)
809
810     def send_expansion_command(self, expansion_cmd: str):
811         """Sends a command to the ESP32 expansion board connected to a Tello Talent
812         Use e.g. tello.send_expansion_command("led 255 0 0") to turn the top led red.
813         """
814         cmd = 'EXT {}'.format(expansion_cmd)
815         self.send_control_command(cmd)
816
817     def query_speed(self) -> int:
818         """Query speed setting (cm/s)
819         Returns:
820             int: 1-100
821         """
822         return self.send_read_command_int('speed?')
823
824     def query_battery(self) -> int:
825         """Get current battery percentage via a query command
826         Using get_battery is usually faster
827         Returns:
828             int: 0-100 in %
829         """
830         return self.send_read_command_int('battery?')
831
832     def query_flight_time(self) -> int:
833         """Query current fly time (s).
834         Using get_flight_time is usually faster.
835         Returns:
836             int: Seconds elapsed during flight.
837         """
838         return self.send_read_command_int('time?')

```

```
830
831     def query_height(self) -> int:
832         """Get height in cm via a query command.
833         Using get_height is usually faster
834         Returns:
835             int: 0-3000
836             """
837         return self.send_read_command_int('height?')
838
839     def query_temperature(self) -> int:
840         """Query temperature (°C).
841         Using get_temperature is usually faster.
842         Returns:
843             int: 0-90
844             """
845         return self.send_read_command_int('temp?')
846
847     def query_attitude(self) -> dict:
848         """Query IMU attitude data.
849         Using get_pitch, get_roll and get_yaw is usually faster.
850         Returns:
851             {'pitch': int, 'roll': int, 'yaw': int}
852             """
853         response = self.send_read_command('attitude?')
854         return Tello.parse_state(response)
855
856     def query_barometer(self) -> int:
857         """Get barometer value (cm)
858         Using get_barometer is usually faster.
859         Returns:
860             int: 0-100
861             """
862         baro = self.send_read_command_int('baro?')
863         return baro * 100
864
865     def query_distance_tof(self) -> float:
866         """Get distance value from TOF (cm)
867         Using get_distance_tof is usually faster.
868         Returns:
869             float: 30-1000
870             """
871         # example response: 801mm
872         tof = self.send_read_command('tof?')
873         return int(tof[:-2]) / 10
874
875     def query_wifi_signal_noise_ratio(self) -> str:
876         """Get Wi-Fi SNR
877         Returns:
878             str: snr
879             """
880         return self.send_read_command('wifi?')
881
882     def query_sdk_version(self) -> str:
883         """Get SDK Version
884         Returns:
885             str: SDK Version
886             """
887         return self.send_read_command('sdk?')
888
889     def query_serial_number(self) -> str:
```

```
881         """Get Serial Number
882         Returns:
883             str: Serial Number
884         """
885         return self.send_read_command('sn?')
886
887     def query_active(self) -> str:
888         """Get the active status
889         Returns:
890             str
891         """
892         return self.send_read_command('active?')
893
894     def end(self):
895         """Call this method when you want to end the tello object
896         """
897         try:
898             if self.is_flying:
899                 self.land()
900             if self.stream_on:
901                 self.streamoff()
902         except TelloException:
903             pass
904
905         if self.background_frame_read is not None:
906             self.background_frame_read.stop()
907
908         host = self.address[0]
909         if host in drones:
910             del drones[host]
911
912     def __del__(self):
913         self.end()
```

932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982

983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028

change_vs_udp(udp_port)

Change the UDP Port for sending video feed from the drone.

Source code in `djitellopy/tello.py`

```
134 def change_vs_udp(self, udp_port):
135     """Change the UDP Port for sending video feed from the drone.
136     """
137     self.vs_udp_port = udp_port
138     self.send_control_command(f'port 8890 {self.vs_udp_port}')
```

connect(wait_for_state=True)

Enter SDK mode. Call this before any of the control functions.

Source code in `djitellopy/tello.py`

```
532 def connect(self, wait_for_state=True):
533     """Enter SDK mode. Call this before any of the control functions.
534     """
535     self.send_control_command("command")
536
537     if wait_for_state:
538         REPS = 20
539         for i in range(REPS):
540             if self.get_current_state():
541                 t = i / REPS # in seconds
542                 Tello.LOGGER.debug("'connect()' received first state packet after
543                 {} seconds".format(t))
544                 break
545                 time.sleep(1 / REPS)
546
547     if not self.get_current_state():
548         raise TelloException('Did not receive a state packet from the Tello')
```

connect_to_wifi(ssid, password)

Connects to the Wi-Fi with SSID and password. After this command the tello will reboot. Only works with Tello EDUs.

Source code in `djitellopy/tello.py`

```
843 def connect_to_wifi(self, ssid: str, password: str):
844     """Connects to the Wi-Fi with SSID and password.
845     After this command the tello will reboot.
846     Only works with Tello EDUs.
847     """
848     cmd = 'ap {} {}'.format(ssid, password)
849     self.send_control_command(cmd)
```

`curve_xyz_speed(x1, y1, z1, x2, y2, z2, speed)`

Fly to x2 y2 z2 in a curve via x1 y1 z1. Speed defines the traveling speed in cm/s.

- Both points are relative to the current position
- The current position and both points must form a circle arc.
- If the arc radius is not within the range of 0.5-10 meters, it raises an Exception
- x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can both be 0.

Parameters:

Name	Type	Description	Default
<code>x1</code>	<code>int</code>	-500-500	<i>required</i>
<code>x2</code>	<code>int</code>	-500-500	<i>required</i>
<code>y1</code>	<code>int</code>	-500-500	<i>required</i>
<code>y2</code>	<code>int</code>	-500-500	<i>required</i>
<code>z1</code>	<code>int</code>	-500-500	<i>required</i>
<code>z2</code>	<code>int</code>	-500-500	<i>required</i>
<code>speed</code>	<code>int</code>	10-60	<i>required</i>

Source code in djitellopy/tello.py

```

719 def curve_xyz_speed(self, x1: int, y1: int, z1: int, x2: int, y2: int, z2: int,
720 speed: int):
721     """Fly to x2 y2 z2 in a curve via x2 y2 z2. Speed defines the traveling speed in
722     cm/s.
723     - Both points are relative to the current position
724     - The current position and both points must form a circle arc.
725     - If the arc radius is not within the range of 0.5-10 meters, it raises an
726     Exception
727     - x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can
728     both be 0.
729     Arguments:
730         x1: -500-500
731         x2: -500-500
732         y1: -500-500
733         y2: -500-500
734         z1: -500-500
735         z2: -500-500
736         speed: 10-60
737     """
738     cmd = 'curve {} {} {} {} {} {} {} {}'.format(x1, y1, z1, x2, y2, z2, speed)
739     self.send_control_command(cmd)

```

curve_xyz_speed_mid(x1, y1, z1, x2, y2, z2, speed, mid)

Fly to x2 y2 z2 in a curve via x2 y2 z2. Speed defines the traveling speed in cm/s.

- Both points are relative to the mission pad with id mid.
- The current position and both points must form a circle arc.
- If the arc radius is not within the range of 0.5-10 meters, it raises an Exception
- x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can both be 0.

Parameters:

Name	Type	Description	Default
x1	int	-500-500	required
y1	int	-500-500	required

z1	int	-500-500	required
x2	int	-500-500	required
y2	int	-500-500	required
z2	int	-500-500	required
speed	int	10-60	required
mid	int	1-8	required

Source code in djitellopy/tello.py

```

752 def curve_xyz_speed_mid(self, x1: int, y1: int, z1: int, x2: int, y2: int, z2: int,
753 speed: int, mid: int):
754     """Fly to x2 y2 z2 in a curve via x2 y2 z2. Speed defines the traveling speed in
755 cm/s.
756     - Both points are relative to the mission pad with id mid.
757     - The current position and both points must form a circle arc.
758     - If the arc radius is not within the range of 0.5-10 meters, it raises an
759 Exception
760     - x1/x2, y1/y2, z1/z2 can't both be between -20-20 at the same time, but can
761 both be 0.
762     Arguments:
763         x1: -500-500
764         y1: -500-500
765         z1: -500-500
766         x2: -500-500
767         y2: -500-500
768         z2: -500-500
769         speed: 10-60
770         mid: 1-8
771     """
771     cmd = 'curve {} {} {} {} {} {} {} m{}'.format(x1, y1, z1, x2, y2, z2, speed,
mid)
    self.send_control_command(cmd)

```

disable_mission_pads()

Disable mission pad detection

Source code in `djitellopy/tello.py`

```
794 def disable_mission_pads(self):
795     """Disable mission pad detection
796     """
797     self.send_control_command("moff")
```

`emergency()`

Stop all motors immediately.

Source code in `djitellopy/tello.py`

```
608 def emergency(self):
609     """Stop all motors immediately.
610     """
611     self.send_command_without_return("emergency")
612     self.is_flying = False
```

`enable_mission_pads()`

Enable mission pad detection

Source code in `djitellopy/tello.py`

```
789 def enable_mission_pads(self):
790     """Enable mission pad detection
791     """
792     self.send_control_command("mon")
```

`end()`

Call this method when you want to end the tello object

Source code in `djitellopy/tello.py`

```

1009 def end(self):
1010     """Call this method when you want to end the tello object
1011     """
1012     try:
1013         if self.is_flying:
1014             self.land()
1015         if self.stream_on:
1016             self.streamoff()
1017     except TelloException:
1018         pass
1019
1020     if self.background_frame_read is not None:
1021         self.background_frame_read.stop()
1022
1023     host = self.address[0]
1024     if host in drones:
1025         del drones[host]

```

flip(direction)

Do a flip maneuver. Users would normally call one of the flip_x functions instead.

Parameters:

Name	Type	Description	Default
<code>direction</code>	<code>str</code>	l (left), r (right), f (forward) or b (back)	<i>required</i>

Source code in `djitellopy/tello.py`

```

679 def flip(self, direction: str):
680     """Do a flip maneuver.
681     Users would normally call one of the flip_x functions instead.
682     Arguments:
683         direction: l (left), r (right), f (forward) or b (back)
684     """
685     self.send_control_command("flip {}".format(direction))

```

flip_back()

Flip backwards.

Source code in `djitellopy/tello.py`

```
702 def flip_back(self):
703     """Flip backwards.
704     """
705     self.flip("b")
```

flip_forward()

Flip forward.

Source code in `djitellopy/tello.py`

```
697 def flip_forward(self):
698     """Flip forward.
699     """
700     self.flip("f")
```

flip_left()

Flip to the left.

Source code in `djitellopy/tello.py`

```
687 def flip_left(self):
688     """Flip to the left.
689     """
690     self.flip("l")
```

flip_right()

Flip to the right.

Source code in `djitellopy/tello.py`

```
692 def flip_right(self):
693     """Flip to the right.
694     """
695     self.flip("r")
```

get_acceleration_x()

X-Axis Acceleration

Returns:

Name	Type	Description
<code>float</code>	<code>float</code>	acceleration

Source code in `djitellopy/tello.py`

```
325 def get_acceleration_x(self) -> float:
326     """X-Axis Acceleration
327     Returns:
328         float: acceleration
329     """
330     return self.get_state_field('agx')
```

get_acceleration_y()

Y-Axis Acceleration

Returns:

Name	Type	Description
<code>float</code>	<code>float</code>	acceleration

Source code in djitellopy/tello.py

```
332 def get_acceleration_y(self) -> float:
333     """Y-Axis Acceleration
334     Returns:
335         float: acceleration
336     """
337     return self.get_state_field('agy')
```

get_acceleration_z()

Z-Axis Acceleration

Returns:

Name	Type	Description
float	float	acceleration

Source code in djitellopy/tello.py

```
339 def get_acceleration_z(self) -> float:
340     """Z-Axis Acceleration
341     Returns:
342         float: acceleration
343     """
344     return self.get_state_field('agz')
```

get_barometer()

Get current barometer measurement in cm This resembles the absolute height. See <https://en.wikipedia.org/wiki/Altimeter>

Returns:

Name	Type	Description

`int``int`

barometer measurement in cm

” Source code in `djitellopy/tello.py`

```

383 def get_barometer(self) -> int:
384     """Get current barometer measurement in cm
385     This resembles the absolute height.
386     See https://en.wikipedia.org/wiki/Altimeter
387     Returns:
388         int: barometer measurement in cm
389     """
389     return self.get_state_field('baro') * 100
390

```

`get_battery()`

Get current battery percentage

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	0-100

” Source code in `djitellopy/tello.py`

```

399 def get_battery(self) -> int:
400     """Get current battery percentage
401     Returns:
402         int: 0-100
403     """
404     return self.get_state_field('bat')

```

`get_current_state()`

Call this function to attain the state of the Tello. Returns a dict with all fields. Internal method, you normally wouldn't call this yourself.

Source code in djitellopy/tello.py

```
233 def get_current_state(self) -> dict:
234     """Call this function to attain the state of the Tello. Returns a dict
235     with all fields.
236     Internal method, you normally wouldn't call this yourself.
237     """
238     return self.get_own_udp_object()['state']
```

get_distance_tof()

Get current distance value from TOF in cm

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	TOF distance in cm

Source code in djitellopy/tello.py

```
376 def get_distance_tof(self) -> int:
377     """Get current distance value from TOF in cm
378     Returns:
379         int: TOF distance in cm
380     """
381     return self.get_state_field('tof')
```

get_flight_time()

Get the time the motors have been active in seconds

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	flight time in s

Source code in djitellopy/tello.py

```

392 def get_flight_time(self) -> int:
393     """Get the time the motors have been active in seconds
394     Returns:
395         int: flight time in s
396     """
397     return self.get_state_field('time')

```

get_frame_read(with_queue=False, max_queue_len=32)

Get the BackgroundFrameRead object from the camera drone. Then, you just need to call backgroundFrameRead.frame to get the actual frame received by the drone.

Returns:

Type	Description
BackgroundFrameRead	BackgroundFrameRead

Source code in djitellopy/tello.py

```

413 def get_frame_read(self, with_queue = False, max_queue_len = 32) ->
414     'BackgroundFrameRead':
415     """Get the BackgroundFrameRead object from the camera drone. Then, you just need
416     to call
417     backgroundFrameRead.frame to get the actual frame received by the drone.
418     Returns:
419         BackgroundFrameRead
420     """
421     if self.background_frame_read is None:
422         address = self.get_udp_video_address()
423         self.background_frame_read = BackgroundFrameRead(self, address, with_queue,
max_queue_len)
         self.background_frame_read.start()
         return self.background_frame_read

```

get_height()

Get current height in cm

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	height in cm

” Source code in `djitellopy/tello.py`

```
369 def get_height(self) -> int:
370     """Get current height in cm
371     Returns:
372         int: height in cm
373     """
374     return self.get_state_field('h')
```

`get_highest_temperature()`

Get highest temperature

Returns:

Name	Type	Description
<code>float</code>	<code>int</code>	highest temperature (°C)

” Source code in `djitellopy/tello.py`

```
353 def get_highest_temperature(self) -> int:
354     """Get highest temperature
355     Returns:
356         float: highest temperature (°C)
357     """
358     return self.get_state_field('temph')
```

`get_lowest_temperature()`

Get lowest temperature

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	lowest temperature (°C)

” Source code in `djitellopy/tello.py`

```
346 def get_lowest_temperature(self) -> int:
347     """Get lowest temperature
348     Returns:
349         int: lowest temperature (°C)
350     """
351     return self.get_state_field('templ')
```

`get_mission_pad_distance_x()`

X distance to current mission pad Only available on Tello EDUs after calling `enable_mission_pads`

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	distance in cm

” Source code in `djitellopy/tello.py`

```
259 def get_mission_pad_distance_x(self) -> int:
260     """X distance to current mission pad
261     Only available on Tello EDUs after calling enable_mission_pads
262     Returns:
263         int: distance in cm
264     """
265     return self.get_state_field('x')
```

`get_mission_pad_distance_y()`

Y distance to current mission pad Only available on Tello EDUs after calling enable_mission_pads

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	distance in cm

” Source code in `djitellopy/tello.py`

```
267 def get_mission_pad_distance_y(self) -> int:
268     """Y distance to current mission pad
269     Only available on Tello EDUs after calling enable_mission_pads
270     Returns:
271         int: distance in cm
272     """
273     return self.get_state_field('y')
```

`get_mission_pad_distance_z()`

Z distance to current mission pad Only available on Tello EDUs after calling enable_mission_pads

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	distance in cm

” Source code in `djitellopy/tello.py`

```
275 def get_mission_pad_distance_z(self) -> int:
276     """Z distance to current mission pad
277     Only available on Tello EDUs after calling enable_mission_pads
278     Returns:
279         int: distance in cm
280     """
281     return self.get_state_field('z')
```

get_mission_pad_id()

Mission pad ID of the currently detected mission pad Only available on Tello EDUs after calling `enable_mission_pads`

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	-1 if none is detected, else 1-8

Source code in `djitellopy/tello.py`

```

251 def get_mission_pad_id(self) -> int:
252     """Mission pad ID of the currently detected mission pad
253     Only available on Tello EDUs after calling enable_mission_pads
254     Returns:
255         int: -1 if none is detected, else 1-8
256     """
257     return self.get_state_field('mid')

```

get_own_udp_object()

Get own object from the global drones dict. This object is filled with responses and state information by the receiver threads. Internal method, you normally wouldn't call this yourself.

Source code in `djitellopy/tello.py`

```

140 def get_own_udp_object(self):
141     """Get own object from the global drones dict. This object is filled
142     with responses and state information by the receiver threads.
143     Internal method, you normally wouldn't call this yourself.
144     """
145     global drones
146     host = self.address[0]
147     return drones[host]
148

```

get_pitch()

Get pitch in degree

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	pitch in degree

Source code in `djitellopy/tello.py`

```
283 def get_pitch(self) -> int:
284     """Get pitch in degree
285     Returns:
286         int: pitch in degree
287     """
288     return self.get_state_field('pitch')
```

get_roll()

Get roll in degree

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	roll in degree

Source code in `djitellopy/tello.py`

```
290 def get_roll(self) -> int:
291     """Get roll in degree
292     Returns:
293         int: roll in degree
294     """
295     return self.get_state_field('roll')
```

get_speed_x()

X-Axis Speed

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	speed

” Source code in `djitellopy/tello.py`



```
304 def get_speed_x(self) -> int:
305     """X-Axis Speed
306     Returns:
307         int: speed
308     """
309     return self.get_state_field('vgx')
```

get_speed_y()

Y-Axis Speed

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	speed

” Source code in `djitellopy/tello.py`



```
311 def get_speed_y(self) -> int:
312     """Y-Axis Speed
313     Returns:
314         int: speed
315     """
316     return self.get_state_field('vgy')
```

get_speed_z()

Z-Axis Speed

Returns:

Name	Type	Description
int	int	speed

” Source code in `djitellopy/tello.py`

```
318 def get_speed_z(self) -> int:
319     """Z-Axis Speed
320     Returns:
321         int: speed
322     """
323     return self.get_state_field('vgz')
```

get_state_field(key)

Get a specific sate field by name. Internal method, you normally wouldn't call this yourself.

” Source code in `djitellopy/tello.py`

```
240 def get_state_field(self, key: str):
241     """Get a specific sate field by name.
242     Internal method, you normally wouldn't call this yourself.
243     """
244     state = self.get_current_state()
245
246     if key in state:
247         return state[key]
248     else:
249         raise TelloException('Could not get state property: {}'.format(key))
```

get_temperature()

Get average temperature

Returns:

Name	Type	Description
float	float	average temperature (°C)

” Source code in `djitellopy/tello.py`

```

360 def get_temperature(self) -> float:
361     """Get average temperature
362     Returns:
363         float: average temperature (°C)
364     """
365     templ = self.get_lowest_temperature()
366     temph = self.get_highest_temperature()
367     return (templ + temph) / 2

```

get_udp_video_address()

Internal method, you normally wouldn't call this yourself.

” Source code in `djitellopy/tello.py`

```

406 def get_udp_video_address(self) -> str:
407     """Internal method, you normally wouldn't call this yourself.
408     """
409     address_schema = 'udp://@{ip}:{port}' # + '?overrun_nonfatal=1&fifo_size=5000'
410     address = address_schema.format(ip=self.VS_UDP_IP, port=self.vs_udp_port)
411     return address

```

get_yaw()

Get yaw in degree

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	yaw in degree

Source code in `djitellopy/tello.py`

```
297 def get_yaw(self) -> int:
298     """Get yaw in degree
299     Returns:
300         int: yaw in degree
301     """
302     return self.get_state_field('yaw')
```

`go_xyz_speed(x, y, z, speed)`

Fly to x y z relative to the current position. Speed defines the traveling speed in cm/s.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	-500-500	<i>required</i>
<code>y</code>	<code>int</code>	-500-500	<i>required</i>
<code>z</code>	<code>int</code>	-500-500	<i>required</i>
<code>speed</code>	<code>int</code>	10-100	<i>required</i>

Source code in `djitellopy/tello.py`

```
707 def go_xyz_speed(self, x: int, y: int, z: int, speed: int):
708     """Fly to x y z relative to the current position.
709     Speed defines the traveling speed in cm/s.
710     Arguments:
711         x: -500-500
712         y: -500-500
713         z: -500-500
714         speed: 10-100
715     """
716     cmd = 'go {} {} {} {}'.format(x, y, z, speed)
717     self.send_control_command(cmd)
```

`go_xyz_speed_mid(x, y, z, speed, mid)`

Fly to x y z relative to the mission pad with id mid. Speed defines the traveling speed in cm/s.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	-500-500	<i>required</i>
<code>y</code>	<code>int</code>	-500-500	<i>required</i>
<code>z</code>	<code>int</code>	-500-500	<i>required</i>
<code>speed</code>	<code>int</code>	10-100	<i>required</i>
<code>mid</code>	<code>int</code>	1-8	<i>required</i>

Source code in djitellopy/tello.py

```

739 def go_xyz_speed_mid(self, x: int, y: int, z: int, speed: int, mid: int):
740     """Fly to x y z relative to the mission pad with id mid.
741     Speed defines the traveling speed in cm/s.
742     Arguments:
743         x: -500-500
744         y: -500-500
745         z: -500-500
746         speed: 10-100
747         mid: 1-8
748     """
749     cmd = 'go {} {} {} {} m{}'.format(x, y, z, speed, mid)
750     self.send_control_command(cmd)

```

`go_xyz_speed_yaw_mid(x, y, z, speed, yaw, mid1, mid2)`

Fly to x y z relative to mid1. Then fly to 0 0 z over mid2 and rotate to yaw relative to mid2's rotation. Speed defines the traveling speed in cm/s.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	-500-500	<i>required</i>
<code>y</code>	<code>int</code>	-500-500	<i>required</i>
<code>z</code>	<code>int</code>	-500-500	<i>required</i>
<code>speed</code>	<code>int</code>	10-100	<i>required</i>
<code>yaw</code>	<code>int</code>	-360-360	<i>required</i>
<code>mid1</code>	<code>int</code>	1-8	<i>required</i>
<code>mid2</code>	<code>int</code>	1-8	<i>required</i>

Source code in `djitellopy/tello.py`

```

773 def go_xyz_speed_yaw_mid(self, x: int, y: int, z: int, speed: int, yaw: int, mid1:
774 int, mid2: int):
775     """Fly to x y z relative to mid1.
776     Then fly to 0 0 z over mid2 and rotate to yaw relative to mid2's rotation.
777     Speed defines the traveling speed in cm/s.
778     Arguments:
779         x: -500-500
780         y: -500-500
781         z: -500-500
782         speed: 10-100
783         yaw: -360-360
784         mid1: 1-8
785         mid2: 1-8
786     """
787     cmd = 'jump {} {} {} {} {} m{} m{}'.format(x, y, z, speed, yaw, mid1, mid2)
788     self.send_control_command(cmd)

```

`initiate_throw_takeoff()`

Allows you to take off by throwing your drone within 5 seconds of this command

Source code in `djitellopy/tello.py`

```

564 def initiate_throw_takeoff(self):
565     """Allows you to take off by throwing your drone within 5 seconds of this
566     command
567     """
568     self.send_control_command("throwfly")
569     self.is_flying = True

```

`land()`

Automatic landing.

Source code in `djitellopy/tello.py`

```

578 def land(self):
579     """Automatic landing.
580     """
581     self.send_control_command("land")
582     self.is_flying = False

```

`move(direction, x)`

Tello fly up, down, left, right, forward or back with distance x cm. Users would normally call one of the `move_x` functions instead.

Parameters:

Name	Type	Description	Default
<code>direction</code>	<code>str</code>	up, down, left, right, forward or back	<i>required</i>
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```

614 def move(self, direction: str, x: int):
615     """Tello fly up, down, left, right, forward or back with distance x cm.
616     Users would normally call one of the move_x functions instead.
617     Arguments:
618         direction: up, down, left, right, forward or back
619         x: 20-500
620     """
621     self.send_control_command("{} {}".format(direction, x))

```

`move_back(x)`

Fly x cm backwards.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```
658 def move_back(self, x: int):
659     """Fly x cm backwards.
660     Arguments:
661         x: 20-500
662     """
663     self.move("back", x)
```

`move_down(x)`

Fly x cm down.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```
630 def move_down(self, x: int):
631     """Fly x cm down.
632     Arguments:
633         x: 20-500
634     """
635     self.move("down", x)
```

`move_forward(x)`

Fly x cm forward.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```
651 def move_forward(self, x: int):
652     """Fly x cm forward.
653     Arguments:
654         x: 20-500
655     """
656     self.move("forward", x)
```

`move_left(x)`

Fly x cm left.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```
637 def move_left(self, x: int):
638     """Fly x cm left.
639     Arguments:
640         x: 20-500
641     """
642     self.move("left", x)
```

`move_right(x)`

Fly x cm right.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```

644 def move_right(self, x: int):
645     """Fly x cm right.
646     Arguments:
647         x: 20-500
648     """
649     self.move("right", x)

```

`move_up(x)`

Fly x cm up.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	20-500	<i>required</i>

Source code in `djitellopy/tello.py`

```

623 def move_up(self, x: int):
624     """Fly x cm up.
625     Arguments:
626         x: 20-500
627     """
628     self.move("up", x)

```

`parse_state(state)` `staticmethod`

Parse a state line to a dictionary Internal method, you normally wouldn't call this yourself.

Source code in `djitellopy/tello.py`

```

199 @staticmethod
200 def parse_state(state: str) -> Dict[str, Union[int, float, str]]:
201     """Parse a state line to a dictionary
202     Internal method, you normally wouldn't call this yourself.
203     """
204     state = state.strip()
205     Tello.LOGGER.debug('Raw state data: {}'.format(state))
206
207     if state == 'ok':
208         return {}
209
210     state_dict = {}
211     for field in state.split(';'):
212         split = field.split(':')
213         if len(split) < 2:
214             continue
215
216         key = split[0]
217         value: Union[int, float, str] = split[1]
218
219         if key in Tello.state_field_converters:
220             num_type = Tello.state_field_converters[key]
221             try:
222                 value = num_type(value)
223             except ValueError as e:
224                 Tello.LOGGER.debug('Error parsing state value for {}: {} to {}'.format(key, value, num_type))
225                 Tello.LOGGER.error(e)
226                 continue
227
228         state_dict[key] = value
229
230     return state_dict
231

```

query_active()

Get the active status

Returns:

Type	Description
<code>str</code>	<code>str</code>

Source code in djitellopy/tello.py

```

1002 def query_active(self) -> str:
1003     """Get the active status
1004     Returns:
1005         str
1006     """
1007     return self.send_read_command('active?')

```

query_attitude()

Query IMU attitude data. Using get_pitch, get_roll and get_yaw is usually faster.

Returns:

Type	Description
dict	{'pitch': int, 'roll': int, 'yaw': int}

Source code in djitellopy/tello.py

```

953 def query_attitude(self) -> dict:
954     """Query IMU attitude data.
955     Using get_pitch, get_roll and get_yaw is usually faster.
956     Returns:
957         {'pitch': int, 'roll': int, 'yaw': int}
958     """
959     response = self.send_read_command('attitude?')
960     return Tello.parse_state(response)

```

query_barometer()

Get barometer value (cm) Using get_barometer is usually faster.

Returns:

Name	Type	Description
------	------	-------------

`int``int`

0-100

” Source code in `djitellopy/tello.py`

```

962 def query_barometer(self) -> int:
963     """Get barometer value (cm)
964     Using get_barometer is usually faster.
965     Returns:
966         int: 0-100
967     """
968     baro = self.send_read_command_int('baro?')
969     return baro * 100

```

`query_battery()`

Get current battery percentage via a query command Using `get_battery` is usually faster

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	0-100 in %

” Source code in `djitellopy/tello.py`

```

921 def query_battery(self) -> int:
922     """Get current battery percentage via a query command
923     Using get_battery is usually faster
924     Returns:
925         int: 0-100 in %
926     """
927     return self.send_read_command_int('battery?')

```

`query_distance_tof()`

Get distance value from TOF (cm) Using `get_distance_tof` is usually faster.

Returns:

Name	Type	Description
<code>float</code>	<code>float</code>	30-1000

” Source code in `djitellopy/tello.py`

```
971 def query_distance_tof(self) -> float:
972     """Get distance value from TOF (cm)
973     Using get_distance_tof is usually faster.
974     Returns:
975         float: 30-1000
976     """
977     # example response: 801mm
978     tof = self.send_read_command('tof?')
979     return int(tof[:-2]) / 10
```

`query_flight_time()`

Query current fly time (s). Using `get_flight_time` is usually faster.

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	Seconds elapsed during flight.

” Source code in `djitellopy/tello.py`

```
929 def query_flight_time(self) -> int:
930     """Query current fly time (s).
931     Using get_flight_time is usually faster.
932     Returns:
933         int: Seconds elapsed during flight.
934     """
935     return self.send_read_command_int('time?')
```

query_height()

Get height in cm via a query command. Using get_height is usually faster

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	0-3000

Source code in `djitellopy/tello.py`

```
937 def query_height(self) -> int:
938     """Get height in cm via a query command.
939     Using get_height is usually faster
940     Returns:
941         int: 0-3000
942     """
943     return self.send_read_command_int('height?')
```

query_sdk_version()

Get SDK Version

Returns:

Name	Type	Description
<code>str</code>	<code>str</code>	SDK Version

Source code in `djitellopy/tello.py`

```
988 def query_sdk_version(self) -> str:
989     """Get SDK Version
990     Returns:
991         str: SDK Version
992     """
993     return self.send_read_command('sdk?')
```

query_serial_number()

Get Serial Number

Returns:

Name	Type	Description
<code>str</code>	<code>str</code>	Serial Number

Source code in `djitellopy/tello.py`

```
995 def query_serial_number(self) -> str:
996     """Get Serial Number
997     Returns:
998         str: Serial Number
999     """
1000     return self.send_read_command('sn?')
```

query_speed()

Query speed setting (cm/s)

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	1-100

Source code in `djitellopy/tello.py`

```
914 def query_speed(self) -> int:
915     """Query speed setting (cm/s)
916     Returns:
917         int: 1-100
918     """
919     return self.send_read_command_int('speed?')
```

query_temperature()

Query temperature (°C). Using get_temperature is usually faster.

Returns:

Name	Type	Description
<code>int</code>	<code>int</code>	0-90

Source code in `djitellopy/tello.py`

```
945 def query_temperature(self) -> int:
946     """Query temperature (°C).
947     Using get_temperature is usually faster.
948     Returns:
949         int: 0-90
950     """
951     return self.send_read_command_int('temp?')
```

query_wifi_signal_noise_ratio()

Get Wi-Fi SNR

Returns:

Name	Type	Description
<code>str</code>	<code>str</code>	snr

Source code in `djitellopy/tello.py`

```
981 def query_wifi_signal_noise_ratio(self) -> str:
982     """Get Wi-Fi SNR
983     Returns:
984         str: snr
985     """
986     return self.send_read_command('wifi?')
```

raise_result_error(command, response)

Used to raise an error after an unsuccessful command Internal method, you normally wouldn't call this yourself.

Source code in `djitellopy/tello.py`

```
524 def raise_result_error(self, command: str, response: str) -> bool:
525     """Used to raise an error after an unsuccessful command
526     Internal method, you normally wouldn't call this yourself.
527     """
528     tries = 1 + self.retry_count
529     raise TelloException("Command '{}' was unsuccessful for {} tries. Latest
530     response:\t'{}'".format(command, tries, response))
```

reboot()

Reboots the drone

Source code in `djitellopy/tello.py`

```
859 def reboot(self):
860     """Reboots the drone
861     """
862     self.send_command_without_return('reboot')
```

rotate_clockwise(x)

Rotate x degree clockwise.

Parameters:

Name	Type	Description	Default
x	int	1-360	required

Source code in `djitellopy/tello.py`

```

665 def rotate_clockwise(self, x: int):
666     """Rotate x degree clockwise.
667     Arguments:
668         x: 1-360
669     """
670     self.send_control_command("cw {}".format(x))

```

`rotate_counter_clockwise(x)`

Rotate x degree counter-clockwise.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>int</code>	1-3600	<i>required</i>

Source code in `djitellopy/tello.py`

```

672 def rotate_counter_clockwise(self, x: int):
673     """Rotate x degree counter-clockwise.
674     Arguments:
675         x: 1-3600
676     """
677     self.send_control_command("ccw {}".format(x))

```

`send_command_with_return(command, timeout=RESPONSE_TIMEOUT)`

Send command to Tello and wait for its response. Internal method, you normally wouldn't call this yourself.

**Return**

bool/str: str with response text on success, False when unsuccessful.

**Source code in `djitellopy/tello.py`**

```

425 def send_command_with_return(self, command: str, timeout: int = RESPONSE_TIMEOUT) ->
426 str:
427     """Send command to Tello and wait for its response.
428     Internal method, you normally wouldn't call this yourself.
429     Return:
430         bool/str: str with response text on success, False when unsuccessful.
431         """
432     # Commands very consecutive makes the drone not respond to them.
433     # So wait at least self.TIME_BTW_COMMANDS seconds
434     diff = time.time() - self.last_received_command_timestamp
435     if diff < self.TIME_BTW_COMMANDS:
436         self.LOGGER.debug('Waiting {} seconds to execute command:
437         {}'.format(diff, command))
438         time.sleep(diff)
439
440     self.LOGGER.info("Send command: '{}'.format(command))
441     timestamp = time.time()
442
443     client_socket.sendto(command.encode('utf-8'), self.address)
444
445     responses = self.get_own_udp_object()['responses']
446
447     while not responses:
448         if time.time() - timestamp > timeout:
449             message = "Aborting command '{}'. Did not receive a response after {}
450             seconds".format(command, timeout)
451             self.LOGGER.warning(message)
452             return message
453             time.sleep(0.1) # Sleep during send command
454
455     self.last_received_command_timestamp = time.time()
456
457     first_response = responses.pop(0) # first datum from socket
458     try:
459         response = first_response.decode("utf-8")
460     except UnicodeDecodeError as e:
461         self.LOGGER.error(e)
462         return "response decode error"
463     response = response.rstrip("\r\n")
464
465     self.LOGGER.info("Response {}: '{}'.format(command, response))
466     return response

```

send_command_without_return(command)

Send command to Tello without expecting a response. Internal method, you normally wouldn't call this yourself.

Source code in djitellopy/tello.py

```
465 def send_command_without_return(self, command: str):
466     """Send command to Tello without expecting a response.
467     Internal method, you normally wouldn't call this yourself.
468     """
469     # Commands very consecutive makes the drone not respond to them. So wait at
470     least self.TIME_BTW_COMMANDS seconds
471     self.LOGGER.info("Send command (no response expected): '{}'.format(command))
472     client_socket.sendto(command.encode('utf-8'), self.address)
```

send_control_command(command, timeout=RESPONSE_TIMEOUT)

Send control command to Tello and wait for its response. Internal method, you normally wouldn't call this yourself.

Source code in djitellopy/tello.py

```
474 def send_control_command(self, command: str, timeout: int = RESPONSE_TIMEOUT) ->
475 bool:
476     """Send control command to Tello and wait for its response.
477     Internal method, you normally wouldn't call this yourself.
478     """
479     response = "max retries exceeded"
480     for i in range(0, self.retry_count):
481         response = self.send_command_with_return(command, timeout=timeout)
482         if 'ok' in response.lower():
483             return True
484         self.LOGGER.debug("Command attempt #{i} failed for command: '{}'.format(i,
485 command))
486     self.raise_result_error(command, response)
487     return False # never reached
```

send_expansion_command(expansion_cmd)

Sends a command to the ESP32 expansion board connected to a Tello Talent Use e.g. `tello.send_expansion_command("led 255 0 0")` to turn the top led red.

Source code in `djitellopy/tello.py`

```

907 def send_expansion_command(self, expansion_cmd: str):
908     """Sends a command to the ESP32 expansion board connected to a Tello Talent
909     Use e.g. tello.send_expansion_command("led 255 0 0") to turn the top led red.
910     """
911     cmd = 'EXT {}'.format(expansion_cmd)
912     self.send_control_command(cmd)

```

send_keepalive()

Send a keepalive packet to prevent the drone from landing after 15s

Source code in `djitellopy/tello.py`

```

549 def send_keepalive(self):
550     """Send a keepalive packet to prevent the drone from landing after 15s
551     """
552     self.send_control_command("keepalive")

```

send_rc_control(left_right_velocity, forward_backward_velocity, up_down_velocity, yaw_velocity)

Send RC control via four channels. Command is sent every `self.TIME_BTW_RC_CONTROL_COMMANDS` seconds.

Parameters:

Name	Type	Description	Default

<code>left_right_velocity</code>	<code>int</code>	-100~100 (left/right)	<i>required</i>
<code>forward_backward_velocity</code>	<code>int</code>	-100~100 (forward/backward)	<i>required</i>
<code>up_down_velocity</code>	<code>int</code>	-100~100 (up/down)	<i>required</i>
<code>yaw_velocity</code>	<code>int</code>	-100~100 (yaw)	<i>required</i>

Source code in `djitellopy/tello.py`

```

815 def send_rc_control(self, left_right_velocity: int, forward_backward_velocity: int,
816 up_down_velocity: int,
817 yaw_velocity: int):
818     """Send RC control via four channels. Command is sent every
819 self.TIME_BTW_RC_CONTROL_COMMANDS seconds.
820     Arguments:
821         left_right_velocity: -100~100 (left/right)
822         forward_backward_velocity: -100~100 (forward/backward)
823         up_down_velocity: -100~100 (up/down)
824         yaw_velocity: -100~100 (yaw)
825     """
826     def clamp100(x: int) -> int:
827         return max(-100, min(100, x))
828
829     if time.time() - self.last_rc_control_timestamp >
830 self.TIME_BTW_RC_CONTROL_COMMANDS:
831         self.last_rc_control_timestamp = time.time()
832         cmd = 'rc {} {} {} {}'.format(
833             clamp100(left_right_velocity),
834             clamp100(forward_backward_velocity),
835             clamp100(up_down_velocity),
836             clamp100(yaw_velocity)
837         )
838         self.send_command_without_return(cmd)

```

`send_read_command(command)`

Send given command to Tello and wait for its response. Internal method, you normally wouldn't call this yourself.

” Source code in `djitellopy/tello.py`

```
490 def send_read_command(self, command: str) -> str:
491     """Send given command to Tello and wait for its response.
492     Internal method, you normally wouldn't call this yourself.
493     """
494     response = self.send_command_with_return(command)
495
496     try:
497         response = str(response)
498     except TypeError as e:
499         self.LOGGER.error(e)
500
501     if any(word in response for word in ('error', 'ERROR', 'False')):
502         self.raise_result_error(command, response)
503         return "Error: this code should never be reached"
504
505     return response
506
```

`send_read_command_float(command)`

Send given command to Tello and wait for its response. Parses the response to an integer Internal method, you normally wouldn't call this yourself.

” Source code in `djitellopy/tello.py`

```
516 def send_read_command_float(self, command: str) -> float:
517     """Send given command to Tello and wait for its response.
518     Parses the response to an integer
519     Internal method, you normally wouldn't call this yourself.
520     """
521     response = self.send_read_command(command)
522     return float(response)
```

`send_read_command_int(command)`

Send given command to Tello and wait for its response. Parses the response to an integer Internal method, you normally wouldn't call this yourself.

Source code in djitellopy/tello.py

```

508 def send_read_command_int(self, command: str) -> int:
509     """Send given command to Tello and wait for its response.
510     Parses the response to an integer
511     Internal method, you normally wouldn't call this yourself.
512     """
513     response = self.send_read_command(command)
514     return int(response)

```

set_mission_pad_detection_direction(x)

Set mission pad detection direction. enable_mission_pads needs to be called first. When detecting both directions detecting frequency is 10Hz, otherwise the detection frequency is 20Hz.

Parameters:

Name	Type	Description	Default
x		0 downwards only, 1 forwards only, 2 both directions	<i>required</i>

Source code in djitellopy/tello.py

```

799 def set_mission_pad_detection_direction(self, x):
800     """Set mission pad detection direction. enable_mission_pads needs to be
801     called first. When detecting both directions detecting frequency is 10Hz,
802     otherwise the detection frequency is 20Hz.
803     Arguments:
804         x: 0 downwards only, 1 forwards only, 2 both directions
805     """
806     self.send_control_command("mdirection {}".format(x))

```

**set_network_ports(state_packet_port,
video_stream_port)**

Sets the ports for state packets and video streaming While you can use this command to reconfigure the Tello this library currently does not support non-default ports (TODO!)

Source code in djitellopy/tello.py

```

851 def set_network_ports(self, state_packet_port: int, video_stream_port: int):
852     """Sets the ports for state packets and video streaming
853     While you can use this command to reconfigure the Tello this library currently
854     does not support
855     non-default ports (TODO!)
856     """
857     cmd = 'port {} {}'.format(state_packet_port, video_stream_port)
    self.send_control_command(cmd)

```

set_speed(x)

Set speed to x cm/s.

Parameters:

Name	Type	Description	Default
x	int	10-100	required

Source code in djitellopy/tello.py

```

808 def set_speed(self, x: int):
809     """Set speed to x cm/s.
810     Arguments:
811         x: 10-100
812     """
813     self.send_control_command("speed {}".format(x))

```

set_video_bitrate(bitrate)

Sets the bitrate of the video stream

Use one of the following for the bitrate argument

Tello.BITRATE_AUTO Tello.BITRATE_1MBPS Tello.BITRATE_2MBPS Tello.BITRATE_3MBPS
Tello.BITRATE_4MBPS Tello.BITRATE_5MBPS

Source code in djitellopy/tello.py

```

864 def set_video_bitrate(self, bitrate: int):
865     """Sets the bitrate of the video stream
866     Use one of the following for the bitrate argument:
867         Tello.BITRATE_AUTO
868         Tello.BITRATE_1MBPS
869         Tello.BITRATE_2MBPS
870         Tello.BITRATE_3MBPS
871         Tello.BITRATE_4MBPS
872         Tello.BITRATE_5MBPS
873     """
874     cmd = 'setbitrate {}'.format(bitrate)
875     self.send_control_command(cmd)

```

set_video_direction(direction)

Selects one of the two cameras for video streaming The forward camera is the regular 1080x720 color camera The downward camera is a grey-only 320x240 IR-sensitive camera

Use one of the following for the direction argument

Tello.CAMERA_FORWARD Tello.CAMERA_DOWNWARD

Source code in djitellopy/tello.py

```

896 def set_video_direction(self, direction: int):
897     """Selects one of the two cameras for video streaming
898     The forward camera is the regular 1080x720 color camera
899     The downward camera is a grey-only 320x240 IR-sensitive camera
900     Use one of the following for the direction argument:
901         Tello.CAMERA_FORWARD
902         Tello.CAMERA_DOWNWARD
903     """
904     cmd = 'downvision {}'.format(direction)
905     self.send_control_command(cmd)

```

set_video_fps(fps)

Sets the frames per second of the video stream

 **Use one of the following for the fps argument**

Tello.FPS_5 Tello.FPS_15 Tello.FPS_30

 **Source code in** `djitellopy/tello.py`

```
886 def set_video_fps(self, fps: str):
887     """Sets the frames per second of the video stream
888     Use one of the following for the fps argument:
889         Tello.FPS_5
890         Tello.FPS_15
891         Tello.FPS_30
892     """
892     cmd = 'setfps {}'.format(fps)
893     self.send_control_command(cmd)
894
```

`set_video_resolution(resolution)`

Sets the resolution of the video stream

 **Use one of the following for the resolution argument**

Tello.RESOLUTION_480P Tello.RESOLUTION_720P

 **Source code in** `djitellopy/tello.py`

```
877 def set_video_resolution(self, resolution: str):
878     """Sets the resolution of the video stream
879     Use one of the following for the resolution argument:
880         Tello.RESOLUTION_480P
881         Tello.RESOLUTION_720P
882     """
882     cmd = 'setresolution {}'.format(resolution)
883     self.send_control_command(cmd)
884
```

`set_wifi_credentials(ssid, password)`

Set the Wi-Fi SSID and password. The Tello will reboot afterwards.

Source code in `djitellopy/tello.py`

```
837 def set_wifi_credentials(self, ssid: str, password: str):
838     """Set the Wi-Fi SSID and password. The Tello will reboot afterwards.
839     """
840     cmd = 'wifi {} {}'.format(ssid, password)
841     self.send_control_command(cmd)
```

streamoff()

Turn off video streaming.

Source code in `djitellopy/tello.py`

```
598 def streamoff(self):
599     """Turn off video streaming.
600     """
601     self.send_control_command("streamoff")
602     self.stream_on = False
603
604     if self.background_frame_read is not None:
605         self.background_frame_read.stop()
606         self.background_frame_read = None
```

streamon()

Turn on video streaming. Use `tello.get_frame_read` afterwards. Video Streaming is supported on all tellos when in AP mode (i.e. when your computer is connected to Tello-XXXXXX WiFi network). Currently Tello EDUs do not support video streaming while connected to a WiFi-network.

!!! Note: If the response is 'Unknown command' you have to update the Tello firmware. This can be done using the official Tello app.

” Source code in `djitellopy/tello.py`

```
584 def streamon(self):
585     """Turn on video streaming. Use `tello.get_frame_read` afterwards.
586     Video Streaming is supported on all tello when in AP mode (i.e.
587     when your computer is connected to Tello-XXXXXX WiFi network).
588     Currently Tello EDUs do not support video streaming while connected
589     to a WiFi-network.
590
591     !!! Note:
592         If the response is 'Unknown command' you have to update the Tello
593         firmware. This can be done using the official Tello app.
594     """
595     self.send_control_command("streamon")
596     self.stream_on = True
```

takeoff()

Automatic takeoff.

” Source code in `djitellopy/tello.py`

```
570 def takeoff(self):
571     """Automatic takeoff.
572
573     # Something it takes a looooooot of time to take off and return a succesful
574     takeoff.
575     # So we better wait. Otherwise, it would give us an error on the following
576     calls.
577     self.send_control_command("takeoff", timeout=Tello.TAKEOFF_TIMEOUT)
578     self.is_flying = True
```

turn_motor_off()

Turns off the motor cooling mode

” Source code in `djitellopy/tello.py`

```
559 def turn_motor_off(self):
560     """Turns off the motor cooling mode
561     """
562     self.send_control_command("motoroff")
```

`turn_motor_on()`

Turn on motors without flying (mainly for cooling)

” Source code in `djitellopy/tello.py`

```
554 def turn_motor_on(self):
555     """Turn on motors without flying (mainly for cooling)
556     """
557     self.send_control_command("motoron")
```

`udp_response_receiver()` `staticmethod`

Setup drone UDP receiver. This method listens for responses of Tello. Must be run from a background thread in order to not block the main thread. Internal method, you normally wouldn't call this yourself.

Source code in djitellopy/tello.py

```
150 @staticmethod
151 def udp_response_receiver():
152     """Setup drone UDP receiver. This method listens for responses of Tello.
153     Must be run from a background thread in order to not block the main thread.
154     Internal method, you normally wouldn't call this yourself.
155     """
156     while True:
157         try:
158             data, address = client_socket.recvfrom(1024)
159             address = address[0]
160             Tello.LOGGER.debug('Data received from {} at
client_socket'.format(address))
161
162             if address not in drones:
163                 continue
164
165             drones[address]['responses'].append(data)
166         except Exception as e:
167             Tello.LOGGER.error(e)
168             break
169
170
```

udp_state_receiver() `staticmethod`

Setup state UDP receiver. This method listens for state information from Tello. Must be run from a background thread in order to not block the main thread. Internal method, you normally wouldn't call this yourself.

” Source code in djitellopy/tello.py

```
172 @staticmethod
173 def udp_state_receiver():
174     """Setup state UDP receiver. This method listens for state information from
175     Tello. Must be run from a background thread in order to not block
176     the main thread.
177     Internal method, you normally wouldn't call this yourself.
178     """
179     state_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
180     state_socket.bind("", Tello.STATE_UDP_PORT)
181
182     while True:
183         try:
184             data, address = state_socket.recvfrom(1024)
185             address = address[0]
186             Tello.LOGGER.debug('Data received from {} at
187             state_socket'.format(address))
188
189             if address not in drones:
190                 continue
191
192             data = data.decode('ASCII')
193             drones[address]['state'] = Tello.parse_state(data)
194
195         except Exception as e:
196             Tello.LOGGER.error(e)
197             break
```